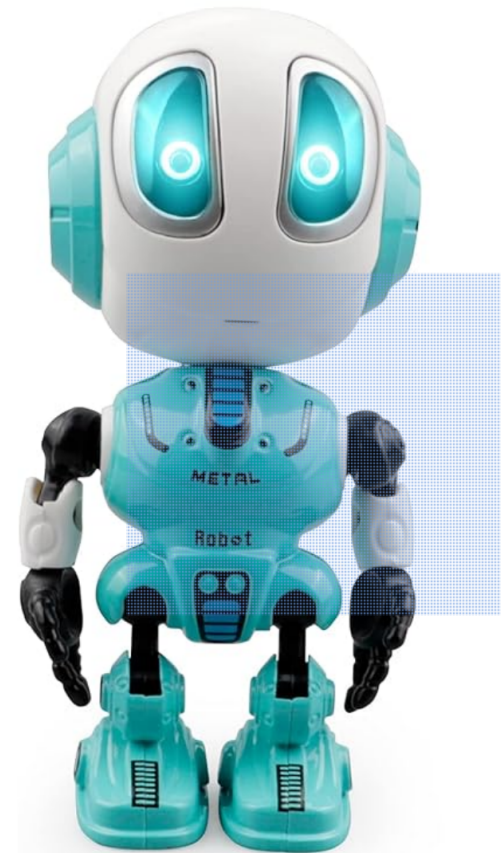


JavaScript Functions

A Comprehensive Guide

JavaScript Functions

- `Alert(leavHause);`
- `Alert(turnLeft);`
- `Alert(turnLeft);`
- `Alert(turnRight);`
- `Alert(turnRight);`
- `Alert(moveUp);`
- `Alert(moveUp);`
- `Alert(moveUp);`
- `Alert(buyMilk);`
- `Alert(moveDown);`
- `Alert(moveDown);`
- `Alert(moveDown);`
- `Alert(turnLeft);`
- `Alert(turnLeft);`
- `Alert(turnRight);`
- `Alert(turnRight);`
- `Alert(enterHause);`



Function

```
function getMilk() {
```

```
    alert("leaveHouse");  
    alert("moveRight");  
    alert("moveRight");  
    alert("moveUp");  
    alert("moveUp");  
    alert("moveUp");  
    alert("moveUp");  
    alert("moveRight");  
    alert("moveRight");  
    alert("buyMilk");  
    alert("moveLeft");  
    alert("moveLeft");  
    alert("moveDown");  
    alert("moveDown");  
    alert("moveDown");  
    alert("moveDown");  
    alert("moveLeft");  
    alert("moveLeft");  
    alert("enterHouse");
```

```
1  getMilk();  
}
```

What is a Function?

- A function is a reusable block of code designed to perform a specific task.
- Helps in organizing code and avoiding repetition.
- Syntax:

```
function functionName() {  
    // Code to be executed  
}
```

- Example:

```
function greet() {  
    console.log("Hello, World!");  
}
```



Why Use Functions?

- **Reusability:** Write once, use multiple times.
- **Modularity:** Break your program into smaller, manageable pieces.
- **Readability:** Easier to understand and debug.
- **Scalability:** Simplifies adding new features.

- Exmample
- <https://stanford.edu/~cpiech/karel/ide.html>

Creating and Calling Functions

- Function Declaration:

```
function sayHello() {  
    console.log("Hello!");  
}
```

- Calling a Function:

```
sayHello(); // Output: Hello!
```

Parameters and Arguments

- **Parameters:** Variables listed in the function declaration.
- **Arguments:** Values passed to the function when it is called.
- Syntax:

```
function greetUser(name) {  
    console.log("Hello, " + name + "!");  
}  
greetUser("Alice"); // Output: Hello, Alice!
```


Example: Multiple Parameters

```
function calculateSum(a, b) {  
    return a + b;  
}  
console.log(calculateSum(5, 10)); // Output: 15
```

Default Parameters

- Default values can be assigned to function parameters.
- Example:

```
function greet(name = "Guest") {  
    console.log("Welcome, " + name + "!");  
}  
  
greet(); // Output: Welcome, Guest!  
greet("John"); // Output: Welcome, John!
```

Returning Values

- - Functions can return a value using the `return` statement.
- - The returned value can be stored in a variable or used directly.
- Syntax:

```
function multiply(a, b) {  
    return a * b;  
}  
  
let result = multiply(4, 5); // Stores 20 in result  
console.log(result); // Output: 20
```

Returning Values Example

```
function getSquare(num) {  
    return num * num;  
}  
console.log(getSquare(6)); // Output: 36
```

Summary

- - Functions make code reusable and modular.
- - Parameters allow passing of dynamic values to functions.
- - The `return` statement provides output from a function.

Exercises

- 1. Write a function that takes two numbers and returns their difference.
- 2. Create a function that calculates the area of a rectangle.
- 3. Write a function that takes a name and age and prints: '<name> is <age> years old.'